

Task 4:

1. State:

- a. Design problem: The valid functionality of a work item is dependent on the state it is currently in. This creates a problem, as a cancelled work item cannot be completed, and if an if/else chain is implemented, it makes the code less readable and causes every method to need updating to add a new state.
- b. Participants:
 - i. State: WorkItemState.
 - ii. ConcreteState: CreatedState, AvailableState, AssignedState, InProgressState, CompletedState, RejectedState, CancelledState.
 - iii. Context: WorkItem.
- c. Collaborate: A WorkItem holds a reference to its current WorkItemState and forwards operations such as start() to it. The state either performs the operation or refuses it and sets the next state on the WorkItem when a transition is allowed.
- d. What varies: The functionality allowed for a work item changes at each state it could be in, as well as the actual available states. (The potential states can be different per process, per scenario, or per organisation).
- e. Why State: Without using State, every work item method would need an if/else chain on a status field to check whether it's valid. With State, each state becomes its own class that decides what each method does; the state class rejects invalid methods, and a new state is an entirely new class.

2. Decorator:

- a. Design problem: Different tasks need different additional functionality, such as validation, priority handling, auditing, notifications, or security, and often multiple at once. Creating a subclass for each combination would cause a large number of classes, and putting every feature in one task class would make it less readable and force unwanted functionality onto tasks that don't need it. The task must still be treated as the same kind of task by the workflow management system.
- b. Participants:
 - i. Component: WorkComponent.
 - ii. ConcreteComponent: ManualTask, AutomatedTask, ApprovalTask.
 - iii. Decorator: WorkComponentDecorator.
 - iv. ConcreteDecorator: ValidatedItem, AuditedItem, NotifyingItem, PriorityItem, SecuredItem.
- c. Collaborate: A decorator holds a WorkComponent, does its extra work before or after forwarding each call to the wrapped component, and implements the same interface so the rest of the system cannot tell the difference. Decorators are stacked when the definition is built.
- d. What varies: The extra behaviours a task has, and in what combination. This can differ from task to task and from organisation to organisation.

- e. Why Decorator: The Decorator pattern wraps the task in a decorator instead of changing the actual task. A decorator implements the same interface as the task it wraps, so the workflow management system treats wrapped and unwrapped tasks the same. Multiple decorators can wrap a task at runtime, so there is no need for a subclass for every combination, and adding new behaviour only requires one new decorator class, with no change to existing task classes.

3. Prototype:

- a. Design problem: A change to one running instance should not accidentally change every other instance created from the same definition. Each instance created from a definition needs its own copy of the structure, data, and progress. Without the Prototype pattern, one way to do this is to rebuild the definition from scratch for every instance, which repeats the construction work and forces the creating code to know every concrete class.
- b. Participants:
 - i. Prototype: WorkComponent.
 - ii. ConcretePrototype: Stage, ManualTask, AutomatedTask, ApprovalTask, WorkComponentDecorator, and its subclasses.
 - iii. PrototypeManager: DefinitionRegistry.
 - iv. Client: WorkflowManager.
- c. Collaborate: DefinitionRegistry stores each finished WorkflowDefinition. When an instance is requested, WorkflowManager asks the registry for the definition and calls clone() on its base WorkComponent. Stage clones each of its children, and decorators clone the component they wrap, so the result is a full deep copy that becomes the new WorkflowInstance. Each applicant gets their own copy of the enrolment tree, so one applicant's progress never changes another's.
- d. What varies: The workflow definition being cloned, which differs per process, and each instance's own data and progress once created. The creating code doesn't depend on the definition's concrete class.
- e. Why Prototype: Builder constructs a workflow definition once. Prototype then clones it, as a deep copy of the whole Composite tree, to create each new instance. Because every stage and work item is copied and none are shared, edits to one instance never affect another or the original definition. The creating code only calls clone(), so it doesn't depend on the definition's concrete class.

4. Composite:

- a. Design problem: A workflow contains stages, which contain individual work items and sub-processes. If single work items and groups of work had different interfaces, every part of the system that handles work would need to check whether it is dealing with one item or a group. Ownership of the work must also be clear.
- b. Participants:
 - i. Component: WorkComponent
 - ii. Leaf: ManualTask, AutomatedTask, ApprovalTask

- iii. Composite: Stage
 - iv. Client: WorkflowInstance
- c. Collaborate: Stage holds a list of WorkComponent children and passes requests like start() and cancel() on to them, in sequence or in parallel depending on its configuration. WorkflowInstance only calls the WorkComponent interface on the root. In enrolment, the root Stage "Enrolment" contains the Stages "Admission", "Fees" and "Registration", and each contains its own work items.
- d. What varies: The tree structure of the work, meaning how deeply it nests, which items are grouped into which stages or sub-processes, and whether a group runs sequentially or in parallel, differs per process.
- e. Why Composite: Single work items (leaves) and groups (composites) implement the same interface, so that the system can start, complete, or query one task or a whole sub-process in the same way, and a group passes the request on to its children. A group owns its children, which makes ownership and lifetime clear. A new kind of stage or work item can be added without changing the code that uses the interface.

5. Iterator:

- a. Design problem: Parts of the system need to go through the work in a workflow, for example, to find the next available item or to show progress, but the work is stored as a nested tree structure. If that code accessed each group's internal storage, it would depend on how groups store their children and would break whenever the storage changed.
- b. Participants:
 - i. Iterator: WorkIterator
 - ii. ConcreteIterator: DepthFirstIterator, ReadyItemIterator
 - iii. Aggregate: WorkComponent
 - iv. ConcreteAggregate: Stage
- c. Collaborate: A client calls createIterator() on a WorkComponent and steps through the result with hasNext() and next(), without seeing how Stage stores its children. WorkflowInstance uses ReadyItemIterator to find items that can become available, and the monitoring code uses DepthFirstIterator to walk the whole tree.
- d. What varies: The traversal order (for example, sequential or depth-first through nested sub-processes) and the internal storage of the groups.
- e. Why Iterator: An iterator gives the client a standard way to traverse through work without knowing how a group stores its children. Each traversal is its own iterator class, so a new way to traverse the structure changes neither the Composite nor its client. Several iterators can be active at once, each keeping its own position.

6. Strategy:

- a. Design problem: Different organisations decide who gets the next task in different ways, ie by role, by current workload, or by a rule. If this logic were written inside the work item classes, those classes would need rewriting, or a conditional chain, for every organisation.

- b. Participants:
 - i. Strategy: AssignmentStrategy
 - ii. ConcreteStrategy: RoleBasedAssignment, WorkloadAssignment, RuleBasedAssignment
 - iii. Context: TaskRouter
- c. Collaborate: When a WorkItem becomes available, TaskRouter calls selectParticipant(item, candidates) on the AssignmentStrategy set for that item in the definition, and assigns the item to the result. TaskRouter never checks which concrete strategy it holds. In enrolment, "assess application" could use RoleBasedAssignment, while a busy period could switch to WorkloadAssignment.
- d. What varies: The rule for how the program chooses who receives the next task. Which rule applies can differ per process or organisation.
- e. Why Strategy: Each rule is its own class behind a common interface, so the part of the system that assigns work calls the rule without knowing which one it is. A new rule is a new class, allowing the work item classes to stay unchanged. The rule can be chosen per workflow definition, and no if/else chain for the type of organisation is needed.

7. Chain of Responsibility:

- a. Design problem: Approval may involve one person, several levels, or escalation when nobody responds. If one approval class held all of these cases, it would contain a conditional chain for each case and need editing whenever an approval route changed.
- b. Participants:
 - i. Handler: ApprovalHandler
 - ii. ConcreteHandler: RoleApprovalHandler, FinalApprovalHandler
 - iii. Client: ApprovalTask
- c. Collaborate: ApprovalTask creates an ApprovalRequest and sends it to the first handler. Each RoleApprovalHandler either handles the request or passes it to its successor, and a missed deadline (reported by DeadlineEscalator from Observer) also passes it on. FinalApprovalHandler handles anything that reaches the end of the chain, so no request is left unhandled. In enrolment, the chain is faculty reviewer, then head of department, then dean, all built from RoleApprovalHandler.
- d. What varies: The number of approval levels, who handles each level, and the conditions under which a request is passed up (for example, rejection, no response by a deadline, or a value above a threshold).
- e. Why Chain of Responsibility: Each approver or level is a handler that either deals with the request or passes it to the next handler. A route with one approver or five is just a chain of varied length. Escalation means passing the request to the next link, and the requester does not need to know who ends up handling it. Changing a route means rearranging handlers, not rewriting logic. The chain needs a final handler so that a request is never left unhandled.

8. Observer:

- a. Design problem: A workflow system must react to what happens around it. Completing one activity may make another available, an external event may trigger new work, and several parts of the system may need to react to the same event. If the work item called each of these directly, it would depend on all of them and need editing whenever another was added.
- b. Participants:
 - i. Subject: EventPublisher
 - ii. ConcreteSubject: WorkflowEventBus
 - iii. Observer: EventListener
 - iv. ConcreteObserver: DependencyActivator, AuditLogger, NotificationSender, MonitoringCollector, DeadlineEscalator
- c. Collaborate: Listeners register with EventPublisher for the event types they care about. When a work item completes, or an external event arrives (for example, "payment received"), the publisher notifies every registered listener. DependencyActivator asks ResourceCoordinator whether dependent items can now become available, AuditLogger records the event, and NotificationSender notifies the applicant.
- d. What varies: The set of interested parties for an event, which can grow or differ per process, and what each one does in response.
- e. Why Observer: The subject announces an event to its registered observers without knowing what they are or what they do. A new interested party is a new observer registered with the subject, with no change to the subject itself. This keeps reactions separate from the core workflow model.

9. Adapter:

- a. Design problem: TIVIDY may need to communicate with external systems (for example, email or payment) whose interfaces were not designed for it. If the engine called each system's interface directly, it would depend on every external interface and need changing whenever one changed.
- b. Participants:
 - i. Adaptee: ExternalInterface
 - ii. Target: Connector
 - iii. Adapter: PaymentConnector, StudentRecordsConnector, MessagingConnector
 - iv. Client: AutomatedTask
- c. Collaborate: AutomatedTask asks ConnectorManager for a Connector by name and calls its execute() operation. The adapter translates that call into the external system's own API calls and converts the response back into the form the engine expects. In enrolment, PaymentConnector turns "charge fee" into the payment gateway's call, and MessagingConnector sends the confirmation message.
- d. What varies: The external system and its interface. Different processes connect to different systems.
- e. Why Adapter: An adapter wraps each external system and presents the one connector interface the workflow management system expects, translating calls between the two. The workflow management system only knows that interface, so a new integration is a new adapter with no change to that

workflow management system. The external interfaces cannot be changed by us, which is why wrapping them is needed.

10. Command:

- a. Design problem: A workflow system must be able to explain what happened. Actions such as assign, approve, reject, and escalate, as well as the state changes they cause, all need to be retained as history and undone if needed. If actions were only method calls, nothing would record them or know how to reverse them.
- b. Participants:
 - i. Command: WorkflowCommand
 - ii. ConcreteCommand: AssignCommand, CompleteCommand, ApproveCommand, RejectCommand, CancelCommand
 - iii. Client: WorkflowManager
 - iv. Invoker: CommandProcessor
 - v. Receiver: WorkItem
- c. Collaborate: WorkflowManager creates a command for the requested action and passes it to CommandProcessor, which executes it on the receiving WorkItem and pushes it onto its history. Calling undo() pops the latest command and reverses it where applicable. In enrolment, undoing RegisterModules or a rejected approval is possible, but undoing "send confirmation message" is not, so that command's undo() refuses.
- d. What varies: The actions performed on work, and what undoing each one means. Some actions may not be undoable.
- e. Why Command: Each action becomes an object that holds the details of what it does, so it can be stored in a history, queued, or undone. The invoker does not need to know which concrete action it runs. A new action is a new command class. Command can also use Memento to hold the state needed for undo.

11. Memento:

- a. Design problem: Rollback and recovery require restoring an earlier state of a workflow instance, for example after a failure. The instance's internal state should not be exposed to the rest of the system just so it can be saved.
- b. Participants:
 - i. Originator: WorkflowInstance
 - ii. Memento: InstanceSnapshot
 - iii. Caretaker: SnapshotHistory
- c. Collaborate: Before a risky step, CommandProcessor asks the WorkflowInstance to createSnapshot(), which copies the state it needs into an InstanceSnapshot, and SnapshotHistory stores it without reading it. If the step fails, the instance is given the snapshot back and restores itself. In enrolment, a snapshot is taken before registration so a failed payment can roll the application back.
- d. What varies: Which state is captured, how much of it, and when (for example, before a risky step or at each state change).

- e. Why Memento: The originator creates a snapshot of its own state, and only the originator can read it back, while the caretaker stores snapshots without seeing inside them. This allows rollback without breaking encapsulation. Snapshots cost memory and time, so they should be taken only when necessary.

12. Template Method:

- a. Design problem: Every workflow instance runs through the same overall execution steps (for example, start, make work available, perform, check the outcome, then complete or handle failure), but the details of some steps differ per process. Copying the skeleton for each process would duplicate the common logic and let the sequences drift apart.
- b. Participants:
 - i. AbstractClass: WorkItem
 - ii. ConcreteClass: ManualTask, AutomatedTask, ApprovalTask
- c. Collaborate: execute() calls a fixed sequence of steps (check that the item is ready, onEnter(), perform(), recordResult()). The subclasses override perform(), so a manual task waits for a person, an automated task calls a Connector, and an approval task sends a request into the chain. Decorators wrap execute() from outside, so the two patterns don't duplicate each other.
- d. What varies: The steps that differ per process, inside an execution order that stays fixed.
- e. Why Template Method: The base class defines the fixed execution skeleton once, and subclasses override only the steps that vary. The order and shared logic live in one place, so every process runs the same way. Note: This pattern has no direct line in the brief, so it must have real varying steps in the final design.

13. Facade:

- a. Design problem: TIVIDY has many subsystems (definitions, instances, routing, events, history, and reporting). If clients called each one directly, they would depend on the internal structure and need to know the correct order of calls.
- b. Participants:
 - i. Facade: WorkflowManager
 - ii. SubsystemClasses: DefinitionRegistry, WorkflowInstance, TaskRouter, EventPublisher, ConnectorManager, CommandProcessor, SnapshotHistory
- c. Collaborate: A client calls operations like createInstance(definitionName), start(instanceId), complete(itemId), and getStatus(instanceId) on WorkflowManager. The manager coordinates the subsystems, such as cloning the definition, wiring the listeners, starting the first items, and running commands, so the client needs no knowledge of the order. The enrolment scenario only talks to WorkflowManager.
- d. What varies: The subsystems behind the entry point, which can change without clients changing.

- e. Why Facade: One class offers simple operations such as create, start, and query a workflow, and handles the coordination between subsystems. Clients depend on that one interface and not on the engine's internals. Subsystems can still be used directly where needed.

14. Mediator:

- a. Design problem: Work items, participants, and resources depend on each other for example, one task waits for another, or participants share a resource. If they referenced each other directly, each class would need to know about the others, and adding a dependency would mean editing many classes.
- b. Participants:
 - i. Mediator: WorkCoordinator
 - ii. ConcreteMediator: ResourceCoordinator
 - iii. Colleague: WorkItem, Participant, Resource
- c. Collaborate: Work items, participants, and shared resources hold a reference to WorkCoordinator and never to each other. When an item is about to become available, DependencyActivator asks the coordinator whether its prerequisites are complete and the resources it needs are free, and ResourceCoordinator answers from its own records. In enrolment, "register for module" can only start once the fee is paid and the module has a free place.
- d. What varies: The dependencies and interaction rules between work items, participants, and resources.
- e. Why Mediator: A mediator holds the coordination logic, and the colleagues talk only to it instead of to each other. Changing how they interact means changing the mediator and not every colleague.

15. Builder:

- a. Design problem: A workflow definition is a complex nested structure of stages, tasks, rules, and dependencies. Constructing it with one large constructor, or by hand in client code, is error-prone, and the same construction process may need to produce different definitions.
- b. Participants:
 - i. Builder: DefinitionBuilder
 - ii. ConcreteBuilder: CodeDefinitionBuilder
 - iii. Director: EnrolmentDirector
 - iv. Product: WorkflowDefinition
- c. Collaborate: The director calls the builder's steps in order (addStage, addTask, setAssignmentStrategy, addDecorator, addDependency, setApprovalChain) and finally calls getDefinition(). The finished WorkflowDefinition is stored in DefinitionRegistry, where Prototype clones it. All enrolment-specific knowledge is in EnrolmentDirector, not in the Workflow management system.
- d. What varies: The contents of the definition (stages, tasks, rules, and dependencies) and the source it is built from.

- e. Why Builder: The builder assembles the definition step by step and returns the finished product, keeping construction separate from the structure itself. The finished definition is then cloned by Prototype to create instances. The Director is only needed if there is a reusable construction sequence.